TrekMesh: an Application for Enhancing Safety in Trekking

Morandini Giulio
Computer Science master degree
giulio.morandini@studenti.unipd.it
id: 2139632

Sut Davide
Computer Science master degree
davide.sut@studenti.unipd.it
id: 2156778

***Abstract*—TrekMesh is an Android application that provides infrastructure-free, peer-to-peer communication for hikers and mountain rescue operations in areas without cellular coverage. Built on Google's Nearby Connections API, it organises nearby smartphones into a delay-tolerant mesh network in which each device acts as both a host and a relay node. The system supports text messaging, image transfer, GPS-stamped SOS alerts, and automated safety features such as a countdown safety timer and virtual breadcrumb tracking. A dual-role architecture distinguishes mobile hiker nodes from fixed mountain-hut nodes, the latter acting as internet gateways for escalating emergencies to civil protection services. Experimental evaluation across four scenarios shows zero packet loss, sub-100 ms median handshake latency, 100% message delivery with ACK round-trip times between 100 and 120 ms, and reliable peer discovery up to 20 m in both Low Power and High Power modes.**

***Index Terms*—mesh network, MANET, Nearby Connections, Bluetooth, Wi-Fi Direct, emergency communication, trekking safety, delay-tolerant network**

I. Introduction

The idea for this project stems from the need to ensure safety and communication continuity in extreme environments, such as mountainous areas or remote zones, where traditional network infrastructures (cellular and internet) are often unavailable, unstable, or completely non-existent. In these scenarios, isolation represents one of the primary risks for hikers and rescue teams, making it difficult to request help or simply coordinate among group members.

The final goal is to create a fully decentralized Delay-Tolerant Mesh network: transforming standard smartphones into active nodes capable of propagating text messages, images and emergency data via a multi-hop peer-to-peer topology.

This report details the architectural choices, the technologies utilized, and the networking strategies adopted to make TrekMesh a reliable tool in critical situations.

II. Motivation and Theoretical Background

A. Motivation

Mountain environments present unique challenges for communication. Hikers and rescue teams operating in remote alpine areas are frequently beyond the reach of cellular infrastructure, which is economically and logistically infeasible to deploy at altitude. As a result, individuals in distress often have no reliable means of raising an alarm even when other hikers are within a few hundred metres. Delayed emergency response is a leading factor in fatal outcomes in mountain accidents, and the inability to communicate between groups hinders both self-organised rescue and coordination with authorities.

At the same time, modern smartphones carry Bluetooth and Wi-Fi radios capable of direct device-to-device communication without any network infrastructure. TrekMesh exploits this capability to provide an offline infrastructure-free communication layer for mountain environments, with a focus on SOS message delivery and low-latency peer-to-peer messaging. Mountain huts (rifugi) will then act as privileged gateway nodes, relaying emergency alerts to civil protection services via the internet when connectivity is available.

B. Mobile Ad-hoc Networks

A MANET is a self-configuring network of mobile devices connected by wireless links, with no fixed infrastructure [1]. Each node acts simultaneously as a host and as a router, forwarding traffic on behalf of others. The key properties are decentralisation (no single point of failure), self-organisation (nodes discover each other automatically), dynamic topology (links appear and disappear as nodes move), and multi-hop routing (messages traverse intermediate nodes to extend range), with some opportunistic network choices, like the one of storing messages until a connection with another device is found.

TrekMesh implements a simplified MANET in which the rifugio node plays the role of a gateway, bridging the ad-hoc network to the internet when connectivity is available, and so mixing it to the concept of Mesh networks.

C. Short-Range Wireless Technologies and the Nearby Connections API

TrekMesh relies on Google's *Nearby Connections API* [2], which abstracts Bluetooth and Wi-Fi Direct into a unified interface for peer discovery and bidirectional encrypted data transfer.

Bluetooth operates in the 2.4 GHz ISM band with an effective range of 10–100 m depending on transmission power and environmental conditions [3]. Wi-Fi Direct extends this range (up to $\sim$200 m in ideal conditions) at the cost of higher energy consumption [4]. The API combines both radios: *Low Power* mode uses Bluetooth only for discovery, while *High Power* mode activates Wi-Fi Direct to increase range and

discovery speed. The API guarantees reliable, in-order delivery through internal acknowledgment and retransmission, which is confirmed empirically by the zero packet loss recorded across all test scenarios.

An important operational constraint is the rate limiting enforced by Google Mobile Services at the OS level: rapid connect/disconnect cycles trigger an exponential backoff that can persist for several minutes. TrekMesh mitigates this with a minimum 3 s settle delay between disconnection and the next scan start.

D. Adaptive Scanning and Energy Management

A persistent tension in always-on mesh networking is the trade-off between discoverability and energy consumption, which is critical for battery-constrained devices far from any charging point. TrekMesh addresses this with an adaptive scanning strategy: the device starts in *Low Power* mode (Bluetooth only) and switches temporarily to *High Power* (BT + Wi-Fi Direct) only when no peers are found within a configurable time window, reverting to *Low Power* once a connection is established, reducing power consumption while preserving the ability to discover peers at greater range.

III. System Requirements and Technological Stack

The development of TrekMesh required a rigorous definition of hardware and software requirements, ensuring operability in critical scenarios.

A. System and Hardware Requirements

The application is designed for Android devices with optimized support starting from Android 12/13+ for granular permission management. From a hardware perspective, the device must integrate three mandatory radio modules:

- **Bluetooth Low Energy (BLE):** Essential for passive beaconing and low-power discovery of neighboring nodes;
- **Wi-Fi Direct (Wi-Fi P2P):** Used for bandwidth scaling during the transfer of heavy files (images) and for high-power deep scans of devices;
- **GNSS Module (GPS/GLONASS/Galileo):** Necessary for geographical timestamping of messages and generating SOS coordinates, with support for altitude tracking via the Fused Location Provider.

B. Authorization and Permissions

Given the nature of the application, the user must grant explicit consent for a series of fundamental permissions:

- `ACCESS_FINE_LOCATION`: For positioning and also scanning Bluetooth/Wi-Fi devices;
- `BLUETOOTH_SCAN` & `ADVERTISE`: For node visibility within the mesh network;
- `NEARBY_WIFI_DEVICES`: Used in newer Android versions to bypass the need for location permissions when utilizing Wi-Fi Direct;
- `POST_NOTIFICATIONS` & `FOREGROUND_SERVICE`: Mandatory to keep safety monitoring active even when the app is in the background or the device is locked.

C. Software Architecture

The software architecture follows Android development standards prioritizing responsiveness and robustness:

- **Language and Concurrency:** Entirely developed in Kotlin, the app utilizes Coroutines and Flows (`StateFlow`/`SharedFlow`) for asynchronous communication between the background network service and the user interface;
- **Networking API:** The core of the P2P communication is built upon the Google Nearby Connections API, configured with the `P2P_CLUSTER` strategy, abstracting the complexity of radio protocol negotiation and managing the handoff between Bluetooth and Wi-Fi;
- **Security:** Every data packet is encrypted using AES-256-GCM before transmission, ensuring message integrity and protecting GPS coordinates against tampering. The current implementation uses a shared symmetric key embedded in the application.

IV. App Description

In this section we describe all the functionalities implemented in the app, justifying the architectural design decisions and motivating them.

A. Mesh Dynamics and P2P Network

From the user's perspective, TrekMesh transforms the device into an active, intelligent relay node within a Delay-Tolerant Network that adapts in real-time to the movements of hikers and the challenges of mountain terrain. When a node moves out of range of a peer, messages are forwarded to any other available peer, effectively propagating information around physical obstacles such as ridges or dense forests. This self-healing capability ensures that as long as a chain of users exists within the area, communication remains possible.

The messaging experience is built on an active propagation model:

- **Proactive Broadcasting:** When a user sends an SOS or an informational update, the device actively broadcasts the data to all discovered peers. These peers store the information and forward it to every new node they encounter;
- **Store-and-Forward Logic:** If a hiker is currently alone, the application holds the outgoing messages in an active buffer. As soon as another hiker (or a *rifugio* gateway) comes into range, the device automatically synchronizes the data;
- **Contextual Awareness:** The application automatically stamps data packets with the sender's last known coordinates and altitude.

B. Adaptive Connectivity Logic

The adaptive connectivity engine of TrekMesh rests on a fundamental assumption: the device must remain a functional mesh node for the entire duration of a multi-day trek, yet

battery life is a critical safety constraint in wilderness environments. The application therefore balances energy consumption against network availability through a duty-cycling strategy.

The device operates primarily in *Low Power* mode, maintaining a continuous but lightweight background presence using Bluetooth Low Energy, allowing it to detect and be detected by immediate neighbours with minimal battery impact. When no peer is found within a configurable time window, the application temporarily escalates to *High Power* mode (BT + Wi-Fi Direct) for a 45-second window to achieve a larger operational radius. Once the window expires, the system reverts to Low Power to preserve energy.

When a user triggers an SOS alert, either manually or via the Safety Timer, the application immediately exits the duty-cycle logic and switches to *High Power* mode for an extended window (approximately 90 seconds). The SOS packet is flagged as high priority in the outgoing queue and is transmitted before any other buffered message as soon as a peer is reached.

The application also performs a continuous low-energy broadcasting mechanism as a unidirectional beacon containing the user's unique ID, current emergency status, and real-time GPS coordinates. This functionality ensures that any other device running TrekMesh, even if the two devices fail to establish a full data connection, can detect sensitive information like the SOS Flare.

C. Communication Protocol and Message Lifecycle

The communication protocol of the application is based on epidemic flooding and data retention, with information travelling via a flood routing mechanism: when a node receives a new message, it automatically replicates and re-broadcasts it to all peers in range. To prevent broadcast spam, the system uses a Time-to-Live (TTL) counter: each message is initialized with a specific number of hops (e.g., 7 for standard messages, 15 for area-wide broadcasts), and every time a peer receives and forwards the message, the TTL is decremented by one. Once the TTL reaches zero, the message is no longer forwarded. Every node also maintains a persistent cache of unique message IDs; if a device receives a message it has already processed, it silently discards it, ensuring that bandwidth is used only for new information.

Since the mesh is decentralized, there is no administrator who can delete a message globally, so we implemented a resolve voting system that allows the community to manage the relevance of information: Any user can mark a received INFO message as "Resolved" (e.g., if a blocked trail is now clear). When the system detects that two distinct users have voted to resolve the same message, it generates a special *DELETE_MSG* packet, which propagates through the mesh with high priority, instructing every node that encounters it to remove the corresponding message from their local database and stop any further forwarding.

As described in Section V, messages are stored in the application to be propagated when a new connection is established. To prevent excessive storage consumption and to ensure that hikers are not distracted by outdated information, messages are automatically purged based on their type and priority:

- **Standard Information:** Routine broadcasts and low-priority INFO messages (P1–P2), such as general trail observations or weather updates, are purged after 6 hours;
- **Emergency and High Priority:** SOS alerts and critical safety information (INFO P3) are retained for 24 hours.

D. Hiker and Rifugio Roles

The system distinguishes between two modalities: the **Hiker**, representing the mobile end-user, and the **Rifugio**, which acts as a high-priority infrastructure gateway.

The Hiker profile is optimized for mobility and privacy, creating a persistent hiker ID (e.g., *Hiker-A3F9*), which is randomly generated and stored locally, without being tied to personal data. It allows other users to recognize the same sender over multiple days of trekking or who the hiker met and how many times, while maintaining anonymity and preventing identity spoofing.

The Rifugio profile transforms the device into a network hub suited for fixed locations with potential internet access. Operators can assign a custom display name (e.g., *"Rifugio Stella Alpina")* that is embedded in all outgoing mesh messages, allowing hikers to identify when they are within range of a managed safety point. When an internet connection is available, the device periodically fetches local weather forecasts and broadcasts them as high-priority area alerts into the mesh (e.g., *"Weather Bulletin [14:30]: Clear, 18°C"*), providing hikers with up-to-date conditions without requiring direct internet access on their devices.

When a Rifugio node receives an SOS message via the mesh, it automatically attempts to forward the alert to emergency authorities via a dedicated HTTP API. They can also manage the lifecycle of the SOS as "Acknowledged" or "Resolved", and propagate back that update through the mesh.

Also, to assist rescuers in understanding a hiker's direction of travel and narrowing down search areas, the app implements a virtual breadcrumbs system: the application automatically records the hiker's GPS coordinates at 15-minute intervals, maintaining a rolling history of the most recent positions. Then, when an alert is triggered, the system automatically attaches the last five breadcrumbs to the outgoing data packet. The hiker profile version also saves in a log file all the IDs of the hikers that are encountered during the trek, associating the time of encounter to each of them. When the hiker connect to a Rifugio, he will send the log file and the Rifugio will store it in a database, associating it with the relative ID of the sender. This process guarantee the possibility to go back up to every hiker that a particular person found, and the possible path that that hiker did.

E. User Interface and Operational Experience

When the app is first launched it presents two pages for correctly guaranteeing all the requirements: the first page is designed to grant all the permissions needed for the application's correct workflow (Location, Bluetooth, and Nearby

Devices), and a second page handles the creation of the user's network identity, selecting if the user will use it in hiker or Rifugio mode.

Then user enters in the real app interface, which is designed with a safety-first philosophy, prioritizing operational efficiency through high-contrast elements and rapid access to emergency functions. The application menu is divided into four relevant functionalities: the main dashboard, the message composer, the detail view of the message, and the settings.

1) Main Dashboard - the Homepage: The homepage serves as the central command center, providing the hiker with some functionalities (as can be seen in figure 1) :

- **Live Mesh Status Bar:** Located at the top of the screen, informs the user of the network's current state, allowing him to know whether an SOS would be transmitted immediately or buffered;
- **Tabbed layout:** The UI uses a tabbed layout to separate "Received" and "Sent" messages, and the setting page. The "Received" tab integrates a live notification system for unread messages that increments as new data packets arrive via the mesh;
- **Emergency SOS Button:** The red action button with the "SOS", available on the main screen. A single tap, if confirmed, immediately broadcasts a high-priority SOS;
- **Message Button:** Under the SOS button, it allows to create and send a custom message trough the message composer;

We also implemented a "safety timer" feature designed for solo hikers or those facing particularly dangerous sections of a trail: performing a long-press on the SOS button, the user can set a countdown (e.g., 30 minutes, 1 hour, or 2 hours) that must be manually checked in by pressing an "I'M OK" button before the countdown expires. If the user fails to check in, potentially due to a fall, injury, or loss of consciousness, the application automatically switches into SOS mode, bypassing all confirmation prompts and immediately broadcasting a high-priority distress signal to the mesh.

2) Message Composer: When the user wants to send a message, the app displays a dedicated full-screen composer designed to structure data for efficient mesh propagation (figure 2):

- **Type and Priority Selectors:** Before typing, the user selects the message category (*INFO*, *SOS* and, only for rifugio nodes, also *BROADCAST*). Each of them can be set with a different priority level (P1–P3), describing the message's visual urgency and its position in the transmission queue of nearby peers;
- **Interactive Image Attachment:** If the user wants to attach also a photo, the app launches a cropping tool, allowing to zoom and rotate the image to focus on critical details (e.g., a specific injury or trail obstruction). To avoid overwhelming the Bluetooth connection, image sharing is performed exclusively over Wi-Fi Direct;
- **GPS Stamping:** The composer automatically fetches the latest GNSS data and embeds the sender's coordinates in the outgoing packet.

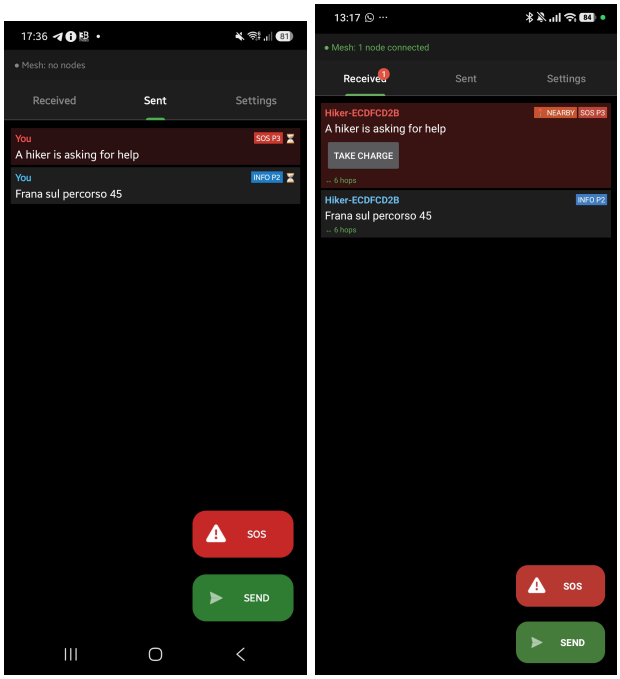

Fig. 1. Homepage (on the left an hiker and on the right a rifugio node)

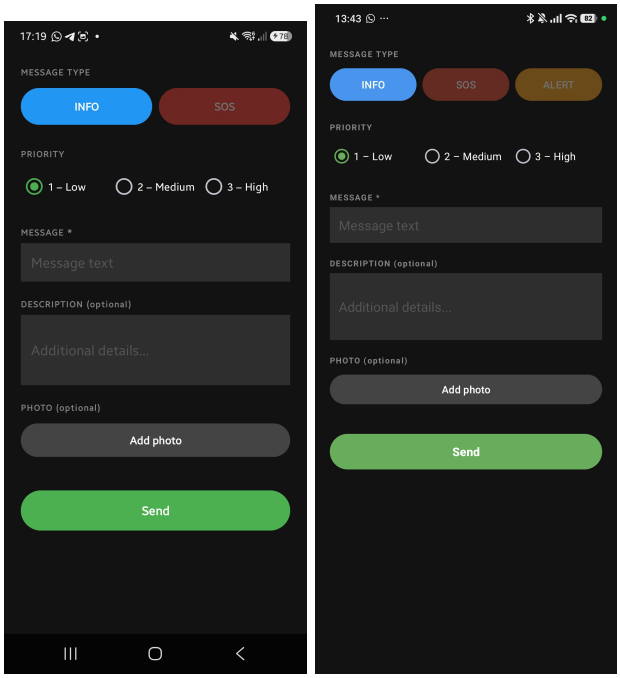

Fig. 2. Message Composer (on the left an hiker and on the right a rifugio node)

3) Detail View: Tapping on a received or sent message displays the extended description like in figure 3, the eventual attached image and metadata such as the sender's identity and the remaining TTL, beyond the "Mark as Resolved" button for received messages and the "Delete" button for sent ones. There is also an integrated link that directly opens the sender's location in any compatible offline mapping application installed

on the device.

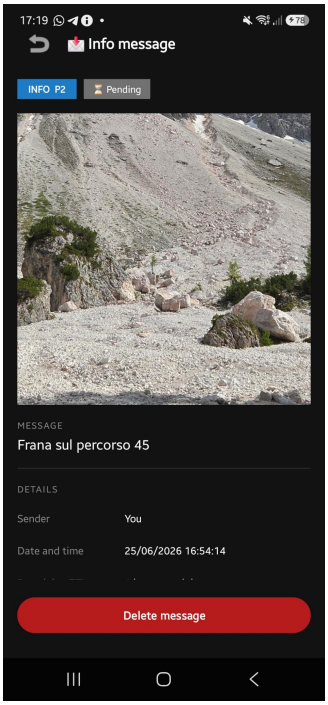

Fig. 3. Example of sent message

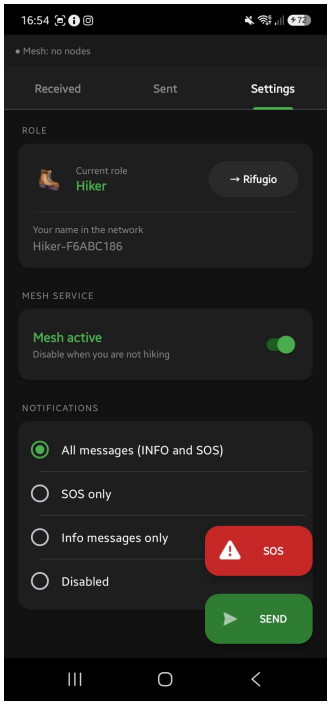

Fig. 4. Settings page

F. Settings

The settings screen allows the user to define their identity and operational mode, enabling or disabling nearby advertising and discovery, switching the role between Hiker and Rifugio and configuring notification filters, as can be seen in figure 4.

V. Experimental Evaluation

A. Functional Testing

Functional tests were conducted with 2–3 Android devices to verify the correct end-to-end behaviour of the application. The following feature categories were validated:

- **Connectivity**: peer discovery, automatic reconnection after disconnection, and multi-device P2P_CLUSTER topology;
- **Messaging**: sending and reception of all message types (INFO, SOS, BROADCAST) with correct priority handling;
- **Mesh propagation**: multi-hop flood routing with correct TTL decrement, deduplication across nodes, and per-type message expiry;
- **SOS lifecycle**: SOS relay to the HTTP API from a rifugio node, acknowledgement and resolution status propagation across the mesh, and safety timer automatic alert on missed check-in;
- **Message lifecycle**: resolve voting with two-user threshold, delete propagation via kill-packet, and correct suppression of already-deleted messages on reconnection;
- **Passive monitoring**: BLE beacon detection of a remote SOS even without an active mesh connection.

All tested features behaved as expected across all scenarios.

B. Performance Test Setup

The performance tests were carried out on two Android devices connected via Bluetooth and Wi-Fi Direct as transport layers. Four experimental scenarios were defined:

- **Close range**: devices approximately 0 - 2 m apart, no obstacles;
- **20 m**: devices approximately 20 m apart in an semi-open environment;
- **40 m**: devices approximately 40 m apart in an semi-open environment;
- **Wall**: a physical wall placed between the devices, spaced out by around 5 meters.

Each test was repeated in automated series run directly by the application, which logs timestamped events for every relevant occurrence (scan start, peer discovery, handshake, data send and receive).

C. Performance Test 1: Discovery Time in Low Power Mode

The first test measures the time required for a node to discover a peer starting from the moment scanning begins in *Low Power* mode (Bluetooth only). The test consists of 10 iterations: in each iteration the node forcibly disconnects and waits to be rediscovered. The scan is started 3 s after disconnection to avoid OS-level rate limiting.

TABLE I
Discovery time in Low Power mode (×10 iterations).

Scenario	Success	Mean	Min	Max
Close range	7/10	31.7 s	3.0 s	81.9 s
20 m	9/10	19.6 s	1.5 s	88.3 s
40 m	3/10	31.9 s	8.4 s	61.1 s
Wall	8/10	23.9 s	3.0 s	66.8 s

Iterations exceeding the 90 s timeout are counted as failures. The results show that at 40 m Low Power mode achieves a success rate of only 30%, highlighting the practical limit of Bluetooth in wide areas.

D. Performance Test 2: Recovery Time after Blackout

The second test simulates a temporary loss of connectivity: the node is taken offline for 10 s and then reactivated. The measured time is the time needed to re-establish the connection with the peer, starting from the moment of reactivation. Reconnection is performed in *High Power* mode (BT+Wi-Fi). The test comprises 5 iterations.

Reconnection succeeded in all iterations across every scenario. The mean recovery time remains below 15 s even in the presence of a physical obstacle, with higher variance in the Wall scenario.

TABLE II
RECOVERY TIME AFTER A 10 S BLACKOUT (×5 ITERATIONS).

Scenario	Success	Mean	Min	Max
Close range	5/5	8.9 s	7.1 s	12.6 s
20 m	5/5	7.5 s	5.1 s	12.2 s
Wall	5/5	14.8 s	4.6 s	28.1 s

E. Performance Test 3: Discovery Time in High Power Mode

Analogous to Test 1, but with *High Power* mode active (Bluetooth + Wi-Fi Direct). This mode incurs higher energy consumption but provides greater range and faster peer discovery.

TABLE III
DISCOVERY TIME IN HIGH POWER MODE (×10 ITERATIONS).

Scenario	Success	Mean	Min	Max
Close range	10/10	8.5 s	2.3 s	12.9 s
20 m	10/10	13.6 s	4.6 s	32.4 s
40 m	5/10	43.2 s	19.7 s	88.3 s
Wall	10/10	12.9 s	2.8 s	26.4 s

Compared to Low Power mode, High Power reduces the mean discovery time by approximately 73% in the close-range scenario (31.7 s $\rightarrow$ 8.5 s). The 40 m scenario remains critical even with High Power, with a 50% success rate and a mean above 40 s.

F. Performance Test 4: Connection Latency (Handshake)

Handshake latency is defined as the time elapsed between the `onConnectionInitiated` event and the corresponding `onConnectionResult` returned by the API. Values were extracted from all log files, including both initial connections and reconnections during test series.

TABLE IV
HANDSHAKE LATENCY EXTRACTED FROM ALL LOGS, PER SCENARIO.

Scenario	Samples	Mean	Median
Close range	22	88 ms	35 ms
20 m	24	62 ms	40 ms
40 m	8	56 ms	60 ms
Wall	22	83 ms	76 ms

Handshake times are in most cases between 20 and 200 ms. We recorded anomalous peaks (>500 ms in the close-range scenario, 9148 ms in the Wall scenario) that are correlated with rate limiting by Google Mobile Services (GMS).

G. Performance Test 5: Packet Loss

This test measures the percentage of lost packets out of 50 messages sent per iteration, over 10 iterations (500 packets total). Reliable transport is guaranteed internally by the Nearby Connections API through an acknowledgment and retransmission mechanism.

No packet loss was recorded in any of the tested scenarios. This result confirms that the Nearby Connections API provides a reliable transport channel regardless of the underlying radio signal quality.

TABLE V
PACKET LOSS OVER 500 PACKETS PER SCENARIO.

Scenario	Packets received	Loss
Close range	500/500	0%
20 m	500/500	0%
Wall	500/500	0%

H. Performance Test 6: Throughput (Image Transfer ∼500 KB)

This test measures the transfer speed of a ∼500 KB payload (500 chunks of ∼1 KB each). Speed is calculated from the moment the first chunk is sent until the last acknowledgment is received. The test comprises 5 iterations per scenario.

TABLE VI
THROUGHPUT PER ITERATION AND SCENARIO (KB/S).

Scenario	Iter 1	Iter 2	Iter 3	Iter 4	Iter 5	Mean
Close range	43.0	41.8	42.0	41.5	44.2	**42.5 KB/s**
20 m	31.5	28.8	29.5	30.5	31.3	**30.3 KB/s**
Wall	44.4	—	—	—	—	**44.4 KB/s***

* Wall scenario: all 500 packets were sent in iterations 2 and 3, but no `THROUGHPUT_PONG` was received (degraded return channel); the physical connection was lost during iteration 4 and iteration 5 was aborted. Only iteration 1 produced a valid measurement.

Throughput settles around 42 KB/s in the close-range scenario, dropping to approximately 30 KB/s at 20 m, a 29% reduction. The Wall scenario result must be considered indicative only, as channel degradation prevented a complete series.

I. Performance Test 7: 10-Message Delivery (ACK Round-Trip Time)

This test measures the round-trip time (RTT) of 10 messages sent in sequence, defined as the time between sending each message and receiving the corresponding acknowledgment from the remote node. Five iterations are executed (50 messages total per scenario).

TABLE VII
MEAN ACK RTT PER ITERATION AND SCENARIO (MS).

Scenario	Iter 1	Iter 2	Iter 3	Iter 4	Iter 5	Mean
Close range	89	117	104	94	94	**100 ms**
20 m	117	83	134	131	137	**120 ms**
Wall	108	111	95	103	106	**104 ms**

The delivery rate is 100% across all scenarios. The mean RTT ranges between 100 and 120 ms with moderate variance. The slightly higher latency at 20 m is attributable to the lower radio channel quality at greater distance.

J. Performance Comparative Summary

Table VIII summaries the main performance indicators across all tested scenarios.

TABLE VIII
SUMMARY OF KEY METRICS PER SCENARIO.

Metric	Close	20 m	40 m	Wall
Disc. LOW (mean)	31.7 s	19.6 s	31.9 s	23.9 s
Disc. LOW (success)	7/10	9/10	3/10	8/10
Recovery 10 s (mean)	8.9 s	7.5 s	—	14.8 s
Disc. HIGH (mean)	8.5 s	13.6 s	43.2 s	12.9 s
Disc. HIGH (success)	10/10	10/10	5/10	10/10
Handshake (median)	35 ms	40 ms	60 ms	76 ms
Packet loss	0%	0%	—	0%
Throughput 500 KB	42.5 KB/s	30.3 KB/s	—	44.4 KB/s*
ACK RTT mean	100 ms	120 ms	—	104 ms
Messages delivered	50/50	50/50	—	50/50

* Partial data: 1 valid iteration out of 5.

K. Battery Consumption Analysis

To evaluate the energy efficiency of TrekMesh, three operational scenarios were measured using AccuBattery on a device running the application in background with the screen off. All figures refer to screen-off drain, to isolate the mesh service overhead from display consumption. Three scenarios were tested: (1) mesh active with one connected peer and no traffic (*Idle mesh*), (2) mesh active with one connected peer receiving messages (*Active messaging*), and (3) no peer in range, adaptive scanning cycling between Low Power and High Power modes (*Adaptive scanning*). The results are summarised in Table IX.

TABLE IX
SCREEN-OFF BATTERY DRAIN ACROSS TREKMESH OPERATIONAL SCENARIOS.

Scenario	Duration	Drain rate	Current (mA)
Idle mesh (1 peer)	71 min	2.3 %/h	114
Active messaging	13 min	3.9 %/h	161
Adaptive scanning	37 min	2.8 %/h	140

The idle mesh scenario exhibits the lowest drain (114 mA, 2.3 %/h), confirming that maintaining a stable Bluetooth connection with a single peer is energy-efficient: on a 5000 mAh battery this projects to approximately 39 hours of continuous background operation. The adaptive scanning scenario (140 mA, 2.8 %/h) incurs a modest 23% overhead compared to the idle baseline, attributable to the periodic *High Power* windows that activate Wi-Fi Direct when no peer is detected. The active messaging scenario shows the highest drain (161 mA, 3.9 %/h). A typical Android device in background standby consumes roughly 1–3 %/h with no active radio tasks, while screen-on usage reaches 10–15 %/h; all three TrekMesh scenarios fall within this latter range or below, demonstrating that the mesh service overhead is sustainable throughout a full-day hike.

An additional qualitative measurement was collected during the performance test sessions described in Section V, where two devices were simultaneously active across close-range, 20–40 m, and wall scenarios, one with active display and the other only using the app in background. Unlike the controlled battery tests above, these sessions involved continuous forced disconnections, automated test series, and active benchmark logging, which explains the higher drain rates observed: the background device reached 8.0 %/h at close range, 7.5 %/h at 20–40 m, and 4.6 %/h through a wall. The foreground device (screen on) reached 14.0 %/h, 15.0 %/h, and 9.2 %/h respectively. These values are consistent with the screen-on baseline and confirm that the elevated drain in those sessions is driven by test activity rather than by the mesh service itself.

VI. CONCLUSIONS AND FUTURE WORKS

TrekMesh demonstrates that commodity Android smartphones can be organised into a functional, infrastructure-free safety network for mountain environments without requiring dedicated hardware or modifications to existing infrastructure. The experimental evaluation confirms zero packet loss across all scenarios, sub-15 s recovery after connectivity blackouts, and reliable peer discovery up to 20 m in both semi-open space and through physical obstacles. These figures are well within the response-time requirements of an emergency alerting system.

The main limitation is the effective range of the Nearby Connections API: at 40 m, discovery success rates drop to 30–50% depending on the scanning mode, constraining the maximum inter-node spacing a deployment can sustain. Future work will focus on evaluating multi-hop message propagation across real trekking routes with more than two nodes, measuring end-to-end latency and delivery rate in dynamic topologies, and formalizing the integration with civil protection alerting systems beyond the current prototype HTTP API.

REFERENCES

[1] S. Corson and J. Macker, "Mobile Ad hoc Networking (MANET): Routing Protocol Performance Issues and Evaluation Considerations," IETF RFC 2501, Jan. 1999.

[2] Google LLC, "Nearby Connections API overview," Android Developers Documentation, 2023. [Online]. Available: https://developers.google.com/nearby/connections/overview

[3] Bluetooth SIG, "Bluetooth Core Specification 5.4," 2023. [Online]. Available: https://www.bluetooth.com/specifications/specs/core54-html/

[4] Wi-Fi Alliance, "Wi-Fi Direct," 2021. [Online]. Available: https://www.wi-fi.org/discover-wi-fi/wi-fi-direct